GUIÓN ACADÉMICO: CÓDIGO JULIA — REGRESIÓN LINEAL NATIVA

`lineal_regresion_guinativa.jl` **— Gradiente descendiente a mano, sin librerías de ML**

🎯 INTRODUCCIÓN GENERAL

Lo que debes decir al empezar a mostrar el código:

> "Este es un programa en Julia que implementa regresión lineal desde cero, sin usar ninguna librería de machine learning. Todo —el modelo, la función de costo, las derivadas, el gradiente descendiente— está escrito a mano. Además tiene una interfaz gráfica con GTK que permite experimentar con diferentes datasets y valores de alfa, y visualizar el entrenamiento en cuatro gráficas interactivas con Plotly."

Estructura del programa (mapa mental):

1. **Importación de librerías** (GUI + graficación).
2. **Datasets predefinidos** (3, 5, 6 y 7 puntos).
3. **Funciones matemáticas centrales** (modelo, costo, derivadas).
4. **Algoritmo de entrenamiento** (gradiente descendiente).
5. **Función de graficación** (4 subplots).
6. **Interfaz gráfica con GTK** (ventana, botones, campos).

📦 LÍNEAS 1-5 — IMPORTS Y CONFIGURACIÓN

```julia
# regresion_lineal.jl -- gradiente descendiente a mano, sin librerias de ML

using Gtk
using Plots
plotlyjs()
```

Qué decir:

> "Importo únicamente dos librerías: **Gtk** para la interfaz gráfica —botones, ventanas, campos de texto— y **Plots** para las gráficas. Notar que NO importo GLM, Flux, ni ninguna librería de machine learning: el algoritmo lo escribo yo mismo. La línea `plotlyjs()` activa PlotlyJS

> como backend, lo que me da gráficas interactivas con zoom, hover sobre los puntos y leyenda clickeable directamente en el navegador."

🔥 **Posibles preguntas:**

P: ¿Por qué PlotlyJS y no otro backend de Plots? R: Julia tiene varios backends (GR, PyPlot, PlotlyJS). GR es el más rápido pero estático. PlotlyJS es más lento pero produce gráficas HTML interactivas, lo cual es crucial aquí para explorar cómo evoluciona la recta iteración por iteración con zoom y hover.

P: ¿Qué es GTK? R: GIMP Toolkit. Es una librería de interfaz gráfica multiplataforma escrita originalmente en C, muy usada en el ecosistema Linux (GNOME). Julia tiene un binding llamado `Gtk.jl` que permite construir ventanas nativas.

P: ¿Por qué no usaste Pluto o una aplicación web? R: Pluto es genial para notebooks reactivos, pero quería una aplicación **standalone**, con ventana nativa del sistema, que no dependiera de un navegador ni de un entorno de notebook. GTK cumple ese rol.

📊 LÍNEAS 7-17 — DATASETS PREDEFINIDOS

```julia
const X_3 = [1.0,2.0,3.0]
const Y_3 = [2.0,4.0,5.5]   # 3 puntos a mano, todos rondan y ~ 1.5x + 0.5

const X_5 = [1.0,2.0,3.0,4.0,5.0]
const Y_5 = [1.5,3.0,4.0,5.5,7.0]

const X_6 = [1.0,2.0,3.0,4.0,5.0,6.0]
const Y_6 = [2.0,2.5,4.0,5.0,6.0,7.5]

const X_7 = [1.0,2.0,3.0,4.0,5.0,6.0,7.0]
const Y_7 = [1.5,3.0,3.5,5.0,5.5,7.0,8.5]
```

Qué decir:

> "Definí cuatro datasets de prueba —de 3, 5, 6 y 7 puntos—. Todos están diseñados para rondar una relación lineal aproximada `y ≈ 1.5x + 0.5`, con algo de ruido manual para hacerlo realista. Uso `const` porque son valores que nunca cambian durante la ejecución; esto le permite al compilador JIT de Julia optimizar más agresivamente."

🔥 **Posibles preguntas:**

P: ¿Por qué usar `const` en Julia? R: `const` le dice al compilador que el tipo y valor no

cambiarán. Esto permite optimizaciones en tiempo de compilación; sin `const`, Julia tendría que volver a verificar el tipo cada vez que accedes a la variable, lo cual penaliza el rendimiento.

P: ¿Por qué los puntos no caen exactamente en una recta? R: Porque si cayeran exactamente, no habría nada que "ajustar". La regresión lineal existe precisamente para manejar datos con ruido y encontrar la **mejor aproximación**. Si los datos fueran perfectos, la solución sería trivial.

P: ¿Y si quisieras probar con otro dataset? R: La interfaz tiene una sección de "entrada manual" (líneas 281-296) donde puedo escribir mis propios valores x e y separados por coma y entrenar con ellos.

P: ¿Por qué empiezas los datos en `1.0` y no en `0.0`? R: Es simplemente una elección didáctica para evitar que el intercepto `b` se confunda con el valor de y en cero. Matemáticamente no hay problema en empezar desde cero.

🔢 LÍNEAS 20-22 — LA FUNCIÓN MODELO

```julia
function modelo(w, b, x)
    return w * x + b
end
```

Qué decir:

> "Esta es la fórmula fundamental del modelo lineal: `f(x) = w·x + b`. Es una línea recta, donde `w` es la pendiente (cuánto sube por cada unidad de x) y `b` es el intercepto (dónde cruza el eje y cuando x = 0). La traducción matemática a código en Julia es literal."

🔥 **Posibles preguntas:**

P: ¿Qué pasa si envías x como array en lugar de un número? R: En esta implementación, la función espera un escalar. Sin embargo, Julia soporta "broadcasting" con el operador punto: `modelo.(w, b, x_rango)` aplica la función a cada elemento. Lo uso en las líneas 91, 135 y 143 para evaluar la recta en varios puntos a la vez.

P: ¿Por qué no usaste multiplicación matricial directa? R: Con una sola variable de entrada, la multiplicación matricial sería exagerada. En problemas con muchas features sí se usa álgebra lineal matricial (`w'x + b`), pero aquí sería sobre-ingeniería para un caso 1D.

P: ¿Qué diferencia hay con la fórmula `y = mx + b` de la escuela? R: Cero matemáticamente. Solo cambia la notación: en machine learning se usa `w` (weight/peso) por convención con redes neuronales, pero es idéntico a `m` (slope/pendiente) de álgebra.

💰 LÍNEAS 25-29 — FUNCIÓN DE COSTO

```julia
function funcion_costo(w, b, x_datos, y_datos)
    m = length(x_datos)
    suma = sum((modelo(w, b, x_datos[i]) - y_datos[i])^2 for i in 1:m)
    return suma / (2.0 * m)
end
```

Qué decir:

> "La función de costo mide **qué tan mal está prediciendo** el modelo. Tomo cada punto de datos, calculo la predicción `modelo(w,b,x_i)`, le resto el valor real `y_i`, elevo al cuadrado, sumo todo, y divido entre `2m`."

Explicación profunda de cada elemento:

- **El cuadrado `^2`:** cumple dos funciones críticas. Primero, penaliza errores grandes más que pequeños (un error de 4 pesa 16). Segundo, impide que errores positivos y negativos se cancelen entre sí.
- **Dividir entre `m`:** hace la métrica comparable entre datasets de distinto tamaño. Es un **promedio**, no una suma.
- **El `1/2`:** es un truco matemático. Al derivar, el exponente 2 baja y se cancela con el 1/2, dejando las fórmulas más limpias (se verá en `dj_dw` y `dj_db`).
- **`sum(... for i in 1:m)`:** es una *generator expression* de Julia. Es eficiente porque no construye un array intermedio, simplemente suma valor por valor.

🔥 Posibles preguntas:

P: ¿Este bucle no hace tu código lento, como pasaría en Python? R: No. Julia tiene compilación JIT (Just-In-Time) especializada: cuando llamo `funcion_costo` con tipos concretos (Float64), el compilador genera código máquina optimizado específicamente para esos tipos. Es casi tan rápido como C.

P: ¿Qué pasa si `m` es cero (dataset vacío)? R: Sería una división por cero y daría `Inf` o `NaN`. No validé ese caso porque confío en que los botones siempre pasan datasets válidos. En código de producción se agregaría una validación `@assert m > 0`.

P: ¿Por qué `2.0` y no `2`? R: Para forzar que la operación sea en punto flotante. En Julia, `2` es entero y `2.0` es Float64. Usar `2.0` evita promociones de tipo innecesarias y deja claro que el resultado será un float.

P: ¿Esto es MSE? R: Casi. Es MSE (Mean Squared Error) dividido a la mitad. Es específicamente la fórmula estándar en cursos de machine learning, donde se incluye el `1/2` por comodidad matemática.

📐 LÍNEAS 32-43 — DERIVADAS PARCIALES

```julia
function dj_dw(w, b, x_datos, y_datos)
    m = length(x_datos)
    suma = sum((modelo(w, b, x_datos[i]) - y_datos[i]) * x_datos[i] for i in 1:m)
    return suma / m
end

function dj_db(w, b, x_datos, y_datos)
    m = length(x_datos)
    suma = sum(modelo(w, b, x_datos[i]) - y_datos[i] for i in 1:m)
    return suma / m
end
```

Qué decir:

> "Estas dos funciones calculan las **derivadas parciales** de la función de costo respecto a cada parámetro. Son las que le dicen al algoritmo **en qué dirección** mover w y b para reducir el error."

Observaciones matemáticas importantes:

- **En `dj_dw` aparece multiplicando por `x_datos[i]`** porque al derivar `(wx + b - y)²` respecto a `w`, por regla de la cadena, el factor externo es `2(wx + b - y)` y el interno es `x`. El 2 se cancela con el 1/2 de J.
- **En `dj_db` no aparece `x`** porque al derivar respecto a `b`, la derivada de `b` es 1, no `x`.
- **Por eso se dividen entre `m` y no entre `2m`:** el 2 del exponente ya se canceló con el 1/2 de J, así que queda solo `/m`.

🔥 Posibles preguntas:

P: ¿Por qué no usaste autodiferenciación como Zygote.jl o ForwardDiff.jl? R: Precisamente porque el objetivo del programa es **entender qué pasa por dentro**. Usar autodiff sería hacer trampa: me daría el mismo resultado sin aprender nada. Escribiéndolo a mano demuestro que entiendo la regla de la cadena y el cálculo diferencial aplicado al problema.

P: Deriva a mano la fórmula de ∂J/∂w para demostrarlo. R: Partimos de `J = (1/2m) Σ (wxᵢ + b − yᵢ)²`. Aplicando la regla de la cadena:
$\partial J/\partial w = (1/2m)\ \Sigma\ 2\cdot(wx_i + b - y_i)\cdot x_i = (1/m)\ \Sigma\ (wx_i + b - y_i)\cdot x_i$
Aquí `(wxᵢ + b)` es exactamente `modelo(w, b, x_datos[i])`, por eso el código escribe `(modelo(...) - y_datos[i]) * x_datos[i]`.

P: ¿Y si tuvieras 100 variables de entrada? R: Tendría 100 derivadas parciales (una por peso), pero todas tienen la misma estructura: `∂J/∂wⱼ = (1/m) Σ (ŷᵢ − yᵢ)·xᵢⱼ`. En la práctica se escribe vectorizado con álgebra lineal.

P: ¿Por qué el gradiente apunta en la dirección de mayor crecimiento y no de mayor descenso? R: Es una propiedad matemática del gradiente: por definición, ∇J apunta hacia donde J crece más rápido. Por eso **restamos** el gradiente (multiplicado por α) para bajar.

🎯 LÍNEAS 46-82 — EL GRADIENTE DESCENDIENTE (corazón del programa)

```julia
```

```julia
function gradiente_descendiente(x_datos, y_datos, alfa, num_iter)
    w = 0.0
    b = 0.0

    hist_costo = Float64[]
    hist_w     = Float64[]
    hist_b     = Float64[]

    for iter in 1:num_iter
        grad_w = dj_dw(w, b, x_datos, y_datos)
        grad_b = dj_db(w, b, x_datos, y_datos)

        temp_w = w - alfa * grad_w
        temp_b = b - alfa * grad_b
        w = temp_w
        b = temp_b

        costo = funcion_costo(w, b, x_datos, y_datos)
        push!(hist_costo, costo)
        push!(hist_w, w)
        push!(hist_b, b)
        ...
    end
    return w, b, hist_costo, hist_w, hist_b
end
```

Qué decir:

> "Este es el algoritmo principal. Arranco con w=0 y b=0 (en el origen), y en cada iteración hago tres cosas: (1) calculo los gradientes con los valores actuales de w y b, (2) actualizo w y b dando un paso en la dirección opuesta al gradiente, y (3) registro el historial para graficar después."

Puntos críticos del código:

1. Actualización simultánea (líneas 63-66):

> "Este es el detalle más importante. Calculo los dos gradientes PRIMERO con los valores viejos, luego creo variables temporales `temp_w` y `temp_b`, y ASÍ actualizo. Si actualizara `w` directamente antes de calcular `dj_db`, la función `dj_db` usaría el `w` nuevo y ya no sería gradiente descendiente puro."

2. Historial (líneas 50-52, 69-71):

> "Guardo `hist_costo`, `hist_w`, `hist_b` para poder graficar después la curva de convergencia, el camino sobre la parábola, y la evolución de la recta. Sin este historial, solo tendría el resultado final sin poder visualizar el proceso."

3. Impresión cada 10%:

> "La línea `iter == 1 || iter % max(1, num_iter ÷ 10) == 0 || iter == num_iter` solo imprime en la primera iteración, cada 10% del total, y en la última. Evita saturar la terminal si entreno con 10,000 iteraciones."

🔥 **Posibles preguntas:**

P: ¿Qué pasa si no inicializas w y b en cero? R: Para regresión lineal **no importa**: la función de costo es convexa, así que desde cualquier punto inicial converge al mismo mínimo global. En redes neuronales sí importa muchísimo, porque tienen múltiples mínimos locales.

P: ¿Por qué guardas todo el historial en memoria? ¿No es costoso? R: Para datasets de 3-7 puntos y 1000 iteraciones, guardar tres arrays de 1000 floats es trivial (~24 KB). En problemas reales con millones de iteraciones se guardaría solo cada N pasos, o directamente en disco. Aquí no es necesaria esa optimización.

P: ¿Qué pasa si haces actualización NO simultánea? R: El algoritmo sigue funcionando en muchos casos, pero técnicamente deja de ser gradiente descendiente puro y se convierte en algo más parecido a Gauss-Seidel. Los caminos de convergencia pueden cambiar. En este código simple probablemente el resultado sería casi idéntico, pero el principio correcto es la actualización simultánea.

P: ¿Qué devuelve la función? R: Una tupla de 5 elementos: `(w_final, b_final, historial_costo, historial_w, historial_b)`. Julia soporta múltiples retornos de manera natural a través de tuplas.

P: ¿Qué significa `Float64[]`? R: Un array vacío de Float64. Es equivalente a `Array{Float64}(undef, 0)` pero más conciso. `push!` agrega elementos dinámicamente.

P: ¿Por qué `push!` lleva signo de exclamación? R: Es una convención de Julia: las funciones que **modifican sus argumentos** llevan `!`. `push!(arr, x)` modifica `arr` agregándole `x`. Es una advertencia visual.

📈 LÍNEAS 85-208 — FUNCIÓN DE GRAFICACIÓN (cuatro subplots)

Qué decir:

> "La función `graficar` construye la figura con cuatro paneles que vemos en la interfaz. Cada panel cuenta una historia distinta del mismo entrenamiento."

Subplot 1 — Regresión lineal final (líneas 93-112)

> "Muestra los datos reales (puntos azules) y la recta óptima final (línea roja). Es la gráfica clásica de regresión. Agrego padding del 20% para que los puntos no queden pegados al borde."

Subplot 2 — Evolución de la recta (líneas 114-148)

> "Muestra el **abanico de rectas** en distintos momentos del entrenamiento. Empiezo con una recta casi horizontal (inicialización en w=0, b=0) y progresivamente rota y se traslada hasta la óptima. Uso una paleta azul degradada para que el usuario vea visualmente el paso del tiempo."

Detalle técnico: si hay demasiadas iteraciones, submuestreo con `paso2 = ceil(Int, length(candidatos) / max_rectas)` para no saturar la leyenda con 100 entradas.

Subplot 3 — Superficie de costo J(w) (líneas 150-179)

> "Esta es **la parábola**. Fijo `b = b*` (el óptimo) y dibujo J como función de w solamente. Es 2D en vez de 3D para que se entienda bien. Los puntos naranjas muestran el **camino** que recorrió el gradiente descendiente, y el diamante verde marca el mínimo alcanzado."

Detalle técnico: evalúo J en 400 valores de w en un rango centrado entre w_inicial y w_óptimo, con un margen extra para ver bien la forma de la parábola.

Subplot 4 — Convergencia del costo (líneas 181-196)

> "Grafica el costo J versus número de iteración. Debería verse como una curva que decae exponencialmente al principio y luego se aplana. Si no se aplana, el modelo no terminó de entrenar. Si sube, el α es demasiado grande."

🔥 **Posibles preguntas:**

P: ¿Por qué fijas `b = b*` en el subplot 3 en lugar de graficar la superficie 3D completa? R: Graficar J(w,b) en 3D es un tazón, pero es difícil de interpretar en una imagen 2D. Al fijar b en su valor óptimo, obtengo un "corte" de la superficie que es una parábola 2D, mucho más fácil de entender. La trayectoria de los puntos es una proyección, pero funciona visualmente.

P: ¿Qué es `cgrad(:blues, ...)`? R: Crea una escala de color degradada usando la paleta "blues" de Plots. Permite dar a cada recta del abanico un color progresivamente más oscuro, reforzando visualmente la idea de "tiempo".

P: ¿Qué pasa si meto 5 puntos en `paso_recta = 10`? R: La línea 116 maneja ese caso: `isempty(candidatos) && (candidatos = [n])`. Si no hay suficientes iteraciones para dibujar al ritmo pedido, simplemente dibuja solo la recta final. Es una protección contra divisiones inválidas.

P: ¿Por qué usas `scatter!` con `!`? R: En Plots.jl, las funciones con `!` **modifican la figura existente** en lugar de crear una nueva. `scatter(...)` crea; `scatter!(...)` agrega al plot previo. Esto me permite superponer datos + recta en el mismo gráfico.

P: ¿Por qué el layout es (2,2)? R: Coloca los 4 subplots en una cuadrícula de 2 filas por 2 columnas. Es el layout estándar para un "dashboard" de 4 paneles.

🖥️ LÍNEAS 211-385 — INTERFAZ GRÁFICA CON GTK

Qué decir:

> "El resto del código construye la ventana interactiva. Uso GTK para crear widgets nativos: una ventana principal, frames agrupadores, campos de texto para parámetros, y botones para los datasets predefinidos y entrada manual."

Estructura de la interfaz:

1. **Título** con la fórmula del modelo y la función de costo (líneas 221-227).
2. **Frame de parámetros** con 4 campos: alfa, iteraciones, paso de rectas y máximo de rectas (líneas 242-261).
3. **Frame de datasets predefinidos** con 4 botones (líneas 263-279).
4. **Frame de entrada manual** con dos campos para x y y separados por coma (líneas 281-296).
5. **Frame de resultados** que se actualiza al terminar el entrenamiento (líneas 298-305).

Mecánica de los botones:

> "Cada botón usa `signal_connect` para asociar una función callback al evento `clicked`. Cuando el usuario presiona el botón, se ejecuta `entrenar_y_graficar` con el dataset correspondiente."

La función `entrenar_y_graficar` (líneas 332-342):

> "Es el flujo completo envuelto en `try...catch`: lee parámetros, entrena, muestra resultados en el panel de texto, y grafica. Si algo falla (por ejemplo, el usuario escribe un alfa inválido), el error se muestra en el label de resultados en lugar de crashear la aplicación."

El cierre (líneas 377-384):

```julia
```

```julia
if !isinteractive()
    condicion = Condition()
    signal_connect(ventana, "destroy") do _w
        notify(condicion)
    end
    @async Gtk.gtk_main()
    wait(condicion)
end
```

> "Este bloque es un detalle importante para ejecutar el archivo desde terminal. Si no estoy en el REPL de Julia, bloqueo el hilo principal hasta que el usuario cierre la ventana; de lo contrario, el programa terminaría inmediatamente sin esperar interacción."

🔥 **Posibles preguntas:**

P: ¿Por qué un helper `fila_campo` en lugar de construir cada fila a mano? R: Para evitar repetir código (principio DRY, Don't Repeat Yourself). Tengo 6 filas con la misma estructura (label + input), así que factorizo en una función helper. Si después quisiera cambiar el estilo global, cambio una sola función.

P: ¿Qué hace `set_gtk_property!`? R: Modifica una propiedad de un widget GTK. Por ejemplo, `set_gtk_property!(ent, :text, "0.01")` cambia el texto del input a "0.01". El `!` indica modificación, y el símbolo `:text` es el nombre de la propiedad.

P: ¿Qué pasa si el usuario escribe algo inválido como "abc" en el campo alfa? R: `parse(Float64, "abc")` lanza una excepción. Pero la función está dentro de `try...catch` (líneas 333-341), así que el error se captura y se muestra en el panel "error: <mensaje>" sin crashear.

P: ¿Es reactiva la interfaz, o hay que hacer click cada vez? R: Hay que hacer click. Podría hacerse reactiva (entrenar automáticamente cuando cambien los parámetros) con signals `changed` en los entries, pero agregaría complejidad sin gran beneficio para un caso educativo.

P: ¿Qué hace `@async`? R: Ejecuta la expresión en una tarea asíncrona (cooperativa). `Gtk.gtk_main()` es el loop de eventos de GTK que bloquea; ponerlo en `@async` lo deja correr en "segundo plano" mientras el hilo principal puede esperar la señal de destroy.

P: ¿Por qué `signal_connect(boton, "clicked") do _ ... end`? R: Es la sintaxis de bloque `do` de Julia. Crea una función anónima y la pasa como callback. El `_` indica "ignoro el primer argumento (el widget que disparó el evento)". Es equivalente a escribir una función aparte y pasarla como parámetro.

🧠 PREGUNTAS GENERALES Y TRAMPA

P: Si GLM existe y está optimizado, ¿para qué escribiste todo esto a mano? R: Por tres razones. **Primera**, pedagogía: entender qué pasa dentro de la caja negra. **Segunda**, control total sobre el proceso (inicialización, criterios de parada, visualización del camino). **Tercera**, porque cuando paso a redes neuronales ya no existe solución cerrada, y el gradiente descendiente es el único camino. Este código es el primer escalón hacia deep learning.

P: ¿Qué pasaría si pongo alfa = 1.0 con el dataset de 7 puntos? R: El modelo diverge. Cada iteración da pasos gigantes que rebotan más allá del mínimo. La curva de convergencia sube en vez de bajar, los valores de w crecen a magnitudes enormes, y eventualmente puede aparecer `NaN` por overflow de Float64.

P: ¿Cómo protegerías el código contra divergencia automáticamente? R: Agregaría una verificación: si `isnan(costo) || isinf(costo)` o si `costo > costo_previo * factor`, interrumpir el entrenamiento con un warning. También podría implementar **gradient clipping**: recortar el gradiente si supera un umbral.

P: ¿Cuál es la complejidad computacional del algoritmo? R: Cada iteración es O(m) donde m es el número de puntos (recorre todos los datos para calcular gradientes). Con N iteraciones, total O(N·m). Para regresión lineal con ecuaciones normales sería $O(m \cdot d^2 + d^3)$ donde d es el número de features, pero se escala mejor con iterativo cuando d es grande.

P: ¿Por qué Julia y no Python + NumPy? R: Tres razones: (1) sintaxis matemática natural, casi idéntica a la fórmula; (2) velocidad casi de C sin necesidad de vectorizar todo como en NumPy; (3) soporte nativo para cálculo científico (multiple dispatch, tipos paramétricos). Para prototipado de algoritmos numéricos es excelente.

P: ¿Funcionaría este código con regresión polinómica? R: No directamente, porque `modelo(w, b, x) = w*x + b` asume relación lineal. Para polinómica habría que generalizar: `modelo(w_vec, b, x) = sum(w_vec[k] * x^k for k in 1:length(w_vec)) + b`. Las derivadas también cambian, pero el esqueleto del gradiente descendiente se conserva.

P: ¿Cómo agregarías regularización (L2/Ridge)? R: En la función de costo sumaría el término `λ * w^2`: `J = (1/2m) Σ error² + λ * w^2`. En `dj_dw` se agregaría `+ 2*λ*w`. Es un cambio local y pequeño, muy elegante.

P: ¿Qué diferencia hay con Stochastic Gradient Descent (SGD)? R: Mi implementación es **batch** gradient descent: calcula el gradiente usando TODOS los datos en cada iteración. SGD usa solo UN punto aleatorio por iteración, lo cual es mucho más rápido por iteración pero con trayectoria más ruidosa. Mini-batch es el punto medio. Para 7 puntos, batch es perfecto.

P: ¿Qué pasa si el usuario ingresa x y y de distinta longitud en la entrada manual? R: La línea 365 valida: `length(xs) == length(ys) || error("la cantidad de valores x e y debe ser igual")`. Lanza un error que el `catch` captura y muestra en el panel.

P: ¿Podrías agregar un botón de "detener entrenamiento"? R: Sí, pero requeriría ejecutar `gradiente_descendiente` en una tarea async con una bandera compartida. Actualmente el entrenamiento bloquea la interfaz durante su ejecución, pero como son milisegundos, no es perceptible.

P: ¿Cuál fue la parte más difícil de implementar? R: (Responde honestamente según tu experiencia. Ejemplos válidos:) "El manejo de la interfaz GTK, porque la documentación es menor que en Python. También el ajuste fino de los paddings y ventanas de zoom en las gráficas, para que todos los casos se vieran bien."

✅ CHECKLIST FINAL PARA DEFENDER EL CÓDIGO

- [ ] Puedo explicar qué hace **cada función** (`modelo`, `funcion_costo`, `dj_dw`, `dj_db`, `gradiente_descendiente`) con sus argumentos y retorno.
- [ ] Entiendo por qué `dj_dw` tiene `x` multiplicando y `dj_db` no.
- [ ] Sé justificar la **actualización simultánea** con `temp_w` y `temp_b`.
- [ ] Puedo explicar qué muestra **cada uno de los 4 subplots** y por qué.
- [ ] Conozco el flujo de la interfaz: botón → `entrenar_y_graficar` → `gradiente_descendiente` → `graficar`.
- [ ] Puedo defender el uso de Julia, GTK y PlotlyJS frente a alternativas.
- [ ] Sé qué hacer si me preguntan "¿por qué no usaste GLM?".

Tip de cierre: Si el profesor te pide ejecutar en vivo, prepárate para mostrar los 4 casos (3 puntos con α=0.01, 7 puntos con α=0.003, 7 puntos con α=0.2, 5 puntos con solo 8 iteraciones). Son los mismos casos de tu presentación PDF y conectan código con teoría.